

Laboratório - Inteligência Artificial

Lógica (Prolog) e Lógica Fuzzy

Lucas Galvano

03 de Setembro de 2026

1 Introdução

Este relatório apresenta a resolução de dois exercícios do laboratório de Programação em Lógica (Prolog) e Lógica Fuzzy: (1) a modelagem da árvore genealógica da Família Silva e das relações de parentesco derivadas dela, em Prolog; e (2) um sistema de inferência fuzzy (Mamdani) para frenagem automática de emergência de um veículo autônomo, incluindo a discussão de um problema de saturação encontrado na implementação inicial. O código-fonte completo (arquivos `.pl` e `.py`) está disponível no repositório do GitHub referenciado na Seção 5; aqui são apresentados apenas os resultados e as discussões pedidas pelo enunciado.

2 Parte 1 — Família Silva (Prolog)

2.1 Árvore genealógica

A partir do enunciado — José e Maria são pais de João e Ana; Ana é mãe de Helena e Joana; Paulo é filho de João; Carlos nasceu da relação entre Helena e Paulo — obtém-se a árvore genealógica da Figura 1.

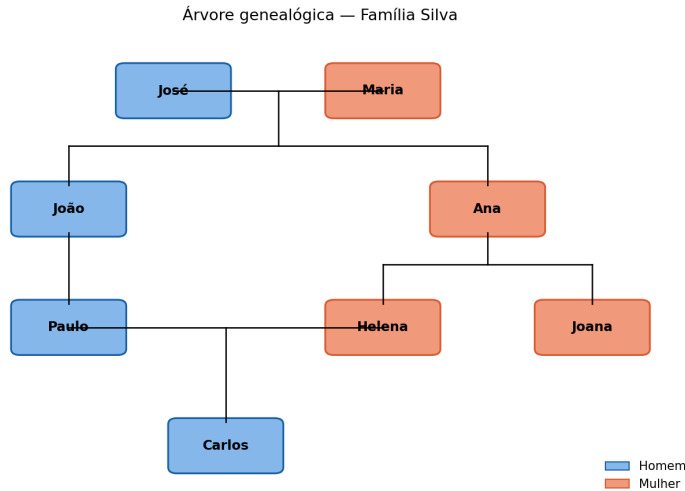


Figura 1: Árvore genealógica da Família Silva.

2.2 Predicados

A base de conhecimento define os fatos `homem/1`, `mulher/1` e `progenitor/2` diretamente a partir do enunciado. A partir deles, foram derivados os predicados `pai/2`, `mae/2`, `irmao/2` e `irma/2` (item c), além dos predicados auxiliares `ascendente/2`, `tio/2`, `tia/2`, `sobrinho/2`, `sobrinha/2`, `primo/2` e `prima/2`, necessários para responder às consultas do item d. O código completo, com os comentários de cada predicado, está no repositório (Seção 5).

2.3 Consultas e resultados

A Tabela 1 apresenta as seis consultas pedidas no item d e os resultados obtidos ao executá-las no SWI-Prolog.

Consulta	Resultado
1. O João é filho do José?	<code>true</code>
2. Quem são os filhos da Maria?	<code>[joao, ana]</code>
3. Quem são os primos do Paulo?	<code>[helena, joana]</code>
4. Sobrinhos/sobrinhas de cada tio/tia	Ana é tia de Paulo; Joana é tia de Carlos; João é tio de Helena e de Joana
5. Quem são os ascendentes do Carlos?	<code>[ana, helena, joao, jose, maria, paulo]</code>
6. A Helena tem irmãos? E irmãs?	Não tem irmãos (homens); tem uma irmã: <code>[joana]</code>

Tabela 1: Consultas do item (d) e resultados obtidos no SWI-Prolog.

Observação sobre a consulta 4

Ao executar a consulta 4 formalmente (`setof(Tio-Sobrinho, sobrinho(Sobrinho,Tio), R)`), o resultado mostra **duas tias** na família — não apenas uma. Além de Ana (tia de Paulo, por ser irmã de João), Joana também é tia de Carlos, pois é irmã de Helena (ambas filhas de Ana), e Helena é mãe de Carlos. Esse é um resultado correto derivado das regras de `tio/2` e `tia/2` definidas — apenas não constava explicitado nos comentários originais do código.

3 Parte 2 — Frenagem de Emergência (Lógica Fuzzy)

3.1 Modelagem

O sistema utiliza duas entradas — distância ao obstáculo (D , 0 a 100 m) e velocidade atual (V , 0 a 100 km/h) — e uma saída, a pressão aplicada no freio (P , 0 a 100%), com funções de pertinência triangulares conforme especificado no enunciado (Figura 2).

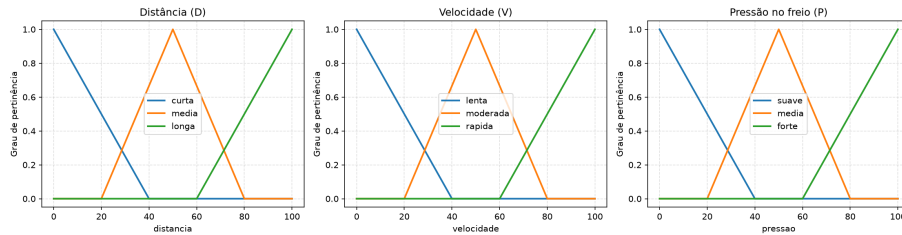


Figura 2: Funções de pertinência de distância, velocidade e pressão no freio.

3.2 Regras de inferência

Foram definidas 7 regras (Mamdani), resumidas na Tabela 2: a regra dominante é que um obstáculo muito próximo exige frenagem forte independentemente da velocidade; nos demais casos, a pressão cresce com a velocidade e diminui com a distância.

Distância \ Velocidade	Lenta	Moderada	Rápida
Curta	Forte	Forte	Forte
Média	Suave	Média	Forte
Longa	Suave	Suave	Média

Tabela 2: Base de regras de inferência (Mamdani).

3.3 Resultados e discussão: o problema da saturação

Ao avaliar o sistema fuzzy, foram analisados casos de teste pontuais em conjunto com a varredura completa de todo o espaço de entrada ($D, V \in [0, 100]$).

A Tabela 3 apresenta a comparação direta entre os métodos de defuzzificação por **Centroide** (padrão) e **MOM** (*Mean of Maximum*) para seis cenários representativos de distância e velocidade.

Distância (m)	Velocidade (km/h)	Pressão - Centróide (%)	Pressão - MOM (%)
10	20	86,67	100,00
10	90	86,67	100,00
50	50	50,00	50,00
90	10	13,33	0,00
90	90	50,00	50,00
50	90	86,67	100,00

Tabela 3: Comparação da pressão de saída obtida nos casos de teste para os métodos Centróide e MOM.

Nota-se claramente pelos casos de teste que, utilizando o método do **Centroide**, mesmo em emergências críticas ($D = 10$ m) a pressão no freio não ultrapassa 86,67%, e em distâncias longas com velocidade baixa ($D = 90$ m, $V = 10$ km/h), a pressão mínima aplicada ainda é de 13,33%. Ao aplicar a defuzzificação por **MOM**, os valores atingem de forma precisa os extremos de 100,00% e 0,00%.

A varredura completa em todo o domínio confirmou este comportamento de saturação:

- **Centroide:** Pressão mínima = 13,33% | Pressão máxima = 86,67%
- **MOM:** Pressão mínima = 0,00% | Pressão máxima = 100,00%

Causa do problema

Isso não é um erro de implementação, e sim uma consequência matemática da combinação entre **funções de pertinência triangulares "retas"** (por exemplo, **forte** = `trimf(60, 100, 100)`, um triângulo com dois vértices coincidentes em 100) e a **defuzzificação por centroide**. Quando uma regra ativa esse conjunto com força total, o centroide do triângulo resultante fica a $2/3$ do caminho entre a base e o pico — não no pico. Para o conjunto **forte** isso dá $60 + \frac{2}{3}(100 - 60) = 86,67$; para o conjunto **suave** (`trimf(0, 0, 40)`), o raciocínio simétrico dá 13,33. A Figura 3 mostra a superfície de saída resultante: nota-se visualmente que o eixo da pressão nunca toca 0 nem 100.

Superfície de saída — defuzzificação por centroide

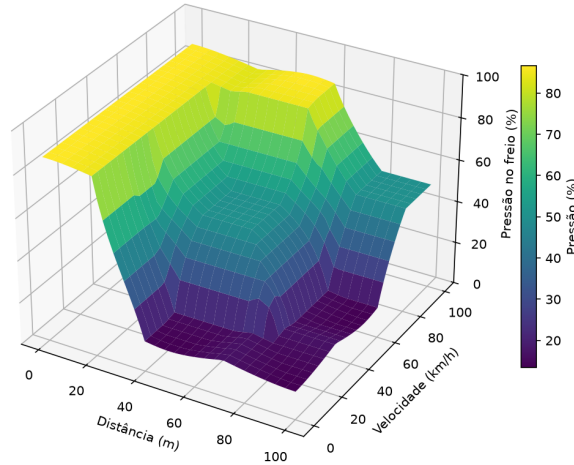


Figura 3: Superfície de saída com defuzzificação por centroide — a pressão fica sempre entre 13,33% e 86,67%.

Solução testada

Duas correções clássicas foram avaliadas:

1. **Alterar as funções de pertinência** para trapézios com platô nas extremidades (por exemplo, `forte = trapmf(60, 90, 100, 100)`). Essa correção teve efeito muito pequeno (o máximo passou de 86,67% para apenas 86,0%), pois para aproximar o centroide de 100 o platô precisaria ser tão largo que o conjunto perderia o caráter "fuzzy" (viraria quase um retângulo).
2. **Trocar o método de defuzzificação** de centroid para mom (*mean of maximum*). Essa alteração — uma única linha de código — resolveu o problema por completo: a nova varredura completa do espaço de entrada resultou em pressão mínima de **0,00%** e máxima de **100,00%**, mantendo o comportamento monotônico esperado nos casos intermediários.

A Figura 4 mostra a superfície de saída após a correção, e a Figura 5 compara diretamente os dois métodos.

Superfície de saída — defuzzificação por MOM (corrigido)

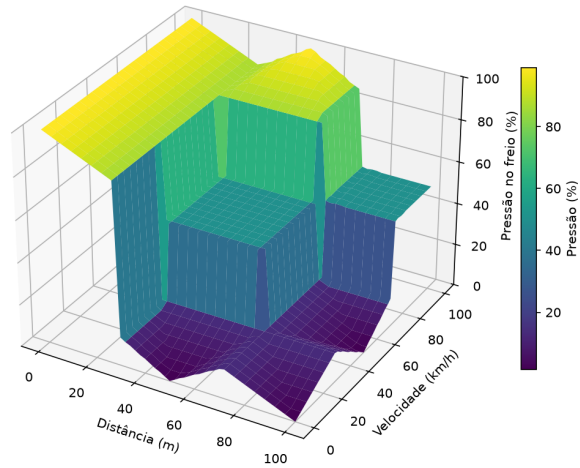


Figura 4: Superfície de saída com defuzzificação por MOM — a pressão agora atinge 0% e 100% nos extremos.

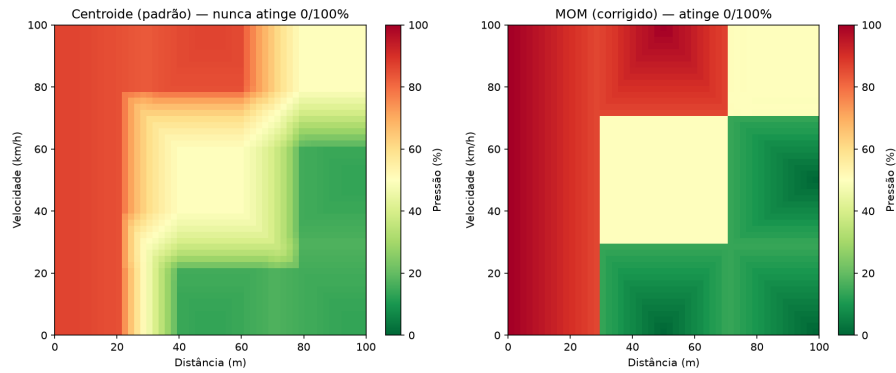


Figura 5: Comparação lado a lado: centroide (esquerda) nunca satura; MOM (direita) satura corretamente nos extremos.

Trade-off entre os métodos

A correção via MOM tem um custo: como o método considera apenas a região de máxima pertinência do conjunto agregado (ignorando a contribuição de conjuntos com ativação parcial), a superfície de saída fica mais "quadriculada", com transições mais abruptas entre regiões — visível na Figura 5. O centroide, por outro lado, produz uma superfície mais suave e contínua (mais adequada ao conforto de frenagem em situações não extremas), mas ao custo de nunca saturar nos limites reais do domínio. Para um sistema de frenagem de emergência, saturar corretamente em 0% e 100% nos casos extremos é o comportamento mais seguro e esperado, o que justifica a escolha do MOM nesse caso.

4 Conclusão

A modelagem em Prolog da Família Silva mostrou como um conjunto pequeno de fatos (`progenitor/2`, `homem/1`, `mulher/1`) permite derivar, por regras simples, relações de parentesco arbitrariamente complexas (tios, sobrinhos, primos, ascendentes) sem necessidade de declará-las explicitamente. Já o exercício de lógica fuzzy evidenciou um ponto importante de engenharia de sistemas fuzzy: a escolha do método de defuzzificação não é um detalhe de implementação, e sim uma decisão de projeto que interage diretamente com o formato das funções de pertinência, podendo levar a comportamentos inesperados de saturação — nesse caso, uma frenagem de emergência que nunca chegava a 100% de pressão, mesmo em uma colisão iminente.

5 Código-fonte

O código-fonte completo (Prolog e Python) está disponível no repositório:

<https://github.com/LucasGalvano/labsIA-CCI620>